NLP LAB EXPERIMENTS

NAME: A. Abhishek Reddy

Rg no:192424192

COURSE CODE: DSA0311

COURSE NAME: NATURAL LANGUAGE PROCESSING

1.Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Aim: To demonstrate how to use regular expressions in Python to match and search for patterns in text.

Algorithm

1. Start.
2. Import the re library.
3. Read the input text.
4. Search and match the required pattern.
5. Display the result.
6. Stop.

CODE:

```
import re
text = "Hello Python! My number is 9876543210."
result = re.search("Python", text)
if result:
    print("Word found:", result.group())
result = re.match("Hello", text)
if result:
    print("Match found:", result.group())
numbers = re.findall(r"\d+", text)
print("Numbers found:", numbers)
```

OUTPUT:

```
Word found: Python
Match found: Hello
Numbers found: ['9876543210']
```

Result: Successfully matched and searched for patterns in text using Python.

2.Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Aim: To implement a basic finite state automaton that accepts strings ending with ab.

Algorithm

1. Start.
2. Read a string.
3. Check whether the string ends with ab.
4. If yes, print Accepted; otherwise, print Rejected.
5. Stop.

CODE:

```
text = input("Enter a string: ")
if text.endswith("ab"):
    print("String Accepted")
else:
    print("String Rejected")
```

OUTPUT:

```
Enter a string: aaab
String Accepted
```

RESULT: Successfully accepted strings ending with ab.

3.Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Aim: To perform morphological analysis using the NLTK library in Python.

Algorithm

1. Start.
2. Import the NLTK stemming and lemmatization libraries.
3. Define a list of words.
4. Apply stemming and lemmatization to each word.
5. Display the results.
6. Stop.

CODE:

```
import nltk
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ["playing", "played", "studies", "running"]
for word in words:
    print("Word:", word)
    print("Stem:", stemmer.stem(word))
    print("Lemma:", lemmatizer.lemmatize(word, pos="v"))
    print()
```

OUTPUT:

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
Word: playing
Stem: play
Lemma: play

Word: played
Stem: play
Lemma: play

Word: studies
Stem: studi
Lemma: study

Word: running
Stem: run
Lemma: run
```

RESULT: Successfully performed stemming and lemmatization using NLTK.

4 Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

Aim: To implement a finite-state machine to generate plural forms of English nouns using Python.

Algorithm

1. Start.
2. Read a singular noun.
3. Check the ending of the noun.
4. Apply the appropriate pluralization rule.
5. Display the plural form.
6. Stop.

CODE:

```
word = input("Enter a noun: ")
if word.endswith(("s", "sh", "ch", "x", "z")):
    plural = word + "es"
elif word.endswith("y") and word[-2] not in "aeiou":
    plural = word[:-1] + "ies"
else:
    plural = word + "s"
print("Plural form:", plural)
```

OUTPUT:

```
Enter a noun: baby
Plural form: babies
```

RESULT: Successfully generated plural forms of English nouns.

5 Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

Aim: To perform word stemming on a list of words using the Porter Stemmer algorithm.

Algorithm

1. Start.
2. Import the Porter Stemmer library.
3. Define a list of words.
4. Apply the Porter Stemmer to each word.
5. Display each word and its stem.
6. Stop.

CODE:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
words = ["playing", "played", "plays", "running", "happily"]
for word in words:
   print(word, "->", ps.stem(word))
```

OUTPUT

```
playing -> play
played -> play
plays -> play
running -> run
happily -> happili
```

RESULT: Successfully reduced words to their stems using the Porter Stemmer algorithm.

6 Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

Aim: To implement a basic N-gram model for text generation using a bigram model in Python.

Algorithm

1. Start.
2. Read the input text.
3. Split the text into words.
4. Form pairs of consecutive words (bigrams).
5. Generate a word sequence using the bigrams.
6. Display the generated text.
7. Stop.

CODE

```
import random
words = "I like Python I like coding".split()
word = "I"
result = [word]
for i in range(4):
    next_words = [words[j+1] for j in range(len(words)-1) if words[j] == word]
    if not next_words:
        break
    word = random.choice(next_words)
    result.append(word)
print("Generated text:", " ".join(result))
```

OUTPUT

```
Generated text: I like Python I like
```

RESULT: The program builds a bigram model from the given text and generates new text by predicting the next word based on the current word.

7 Write program using the NLTK library to perform part-of-speech tagging on a text.

Aim: To perform part-of-speech tagging on a text using the NLTK library in Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Read the input text.
4. Tokenize the text into words.
5. Assign a part-of-speech tag to each word.
6. Display the words and their tags.
7. Stop.

CODE

```
import nltk

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('averaged_perceptron_tagger_eng')

text = "The cat runs fast."

words = nltk.word_tokenize(text)

tags = nltk.pos_tag(words)

print(tags)
```

OUTPUT:

```
[('The', 'DT'), ('cat', 'NN'), ('runs', 'VBZ'), ('fast', 'RB'), ('.', '.')]
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

RESULT: The program successfully identifies the parts of speech of words in the given text.

8 Implement a simple stochastic part-of-speech tagging algorithm using a basic probabilistic model to assign POS tags using python.

Aim: To implement a simple stochastic POS tagging algorithm using a basic probabilistic model in Python.

Algorithm

1. Start.
2. Define a list of words and possible POS tags.
3. Assign a tag to each word using a probabilistic model.
4. Display each word with its assigned tag.
5. Stop.

CODE

```
import random
words = ["I", "love", "Python"]
tags = ["PRP", "VBP", "NN"]
for word in words:
    tag = random.choice(tags)
    print(word, ":", tag)
```

OUTPUT

```
***   I : PRP
      love : NN
      Python : PRP
```

Result: The program demonstrates assigning POS tags using random selection as a simple probabilistic approach.

9 Implement a rule-based part-of-speech tagging system using regular expressions using python.

Aim: To implement a rule-based part-of-speech tagging system using regular expressions in Python.

Algorithm

1. Start.
2. Import the re library.
3. Read the input text.
4. Split the text into words.
5. Apply regular-expression rules to identify word types.
6. Display each word and its tag.

7. Stop.

CODE

```
import re
words = "The cat runs quickly"
for word in words.split():
    if re.match(r".*ly$", word):
        tag = "Adverb"
    elif re.match(r".*ing$", word):
        tag = "Verb"
    elif word in ["cat", "dog"]:
        tag = "Noun"
    else:
        tag = "Other"
    print(word, "-", tag)
```

OUTPUT:

```
The - Other
cat - Noun
runs - Other
quickly - Adverb
```

Result: The program uses regular-expression rules to assign basic parts-of-speech tags to words.

10 Implement transformation-based tagging using a set of transformation rules, apply a simple rule to tag words using python.

Aim: To implement transformation-based tagging using a simple set of rules in Python.

Algorithm

1. Start.
2. Define a list of words.
3. Assign initial tags to the words.
4. Apply transformation rules to change the tags.
5. Display the final tags.

6. Stop.

CODE:

```
words = ["I", "play", "playing", "happy"]
tags = ["Noun", "Noun", "Noun", "Noun"]
for i in range(len(words)):
    if words[i].endswith("ing"):
        tags[i] = "Verb"
    elif words[i] == "I":
        tags[i] = "Pronoun"
for word, tag in zip(words, tags):
    print(word, "-", tag)
```

OUTPUT:

```
I - Pronoun
play - Noun
playing - Verb
happy - Noun
```

Result: The program applies transformation rules to change the initial tags and assign suitable tags to words.